

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

AIML ZG535 — Machine Learning on Edge

LogiEdge: Intelligent Edge AI Platform for Cold-Chain Logistics

Final Report

Git Repo Link : https://github.com/akshay449/ML_on_Edge_Assignment_Group_27

Demo Video Link:

<https://drive.google.com/file/d/1ZikTdX0s3ZPDldi6itOVTneOR2YMXUiY/view?usp=drivesdk>

Group 27	
Linda Kuriakose	2024AC05557
Akshay Ashok Deshpande	2024AD05483
Kamalesh Kumar Gadhwal	2024AC05919
A Jaya Suriya	2024AD05006

Section 1 — FreightBridge Deployment Context

FreightBridge's 85-truck pharmaceutical pilot must detect cargo anomalies within a 90-second SLA while operating through cellular dead zones of 35–90 minutes at seven documented points on the Nashik–Aurangabad route. Four constraints drive the architecture.

Latency: LogiEdge's 10-second window step guarantees a fault is visible to the classifier well inside the 90-second budget, and measured on-device inference is sub-millisecond, so the window cadence — not compute — is the limiting factor; cloud inference cannot meet this SLA at all once a truck enters a coverage gap, since there is no link for a round-trip time to be measured against.

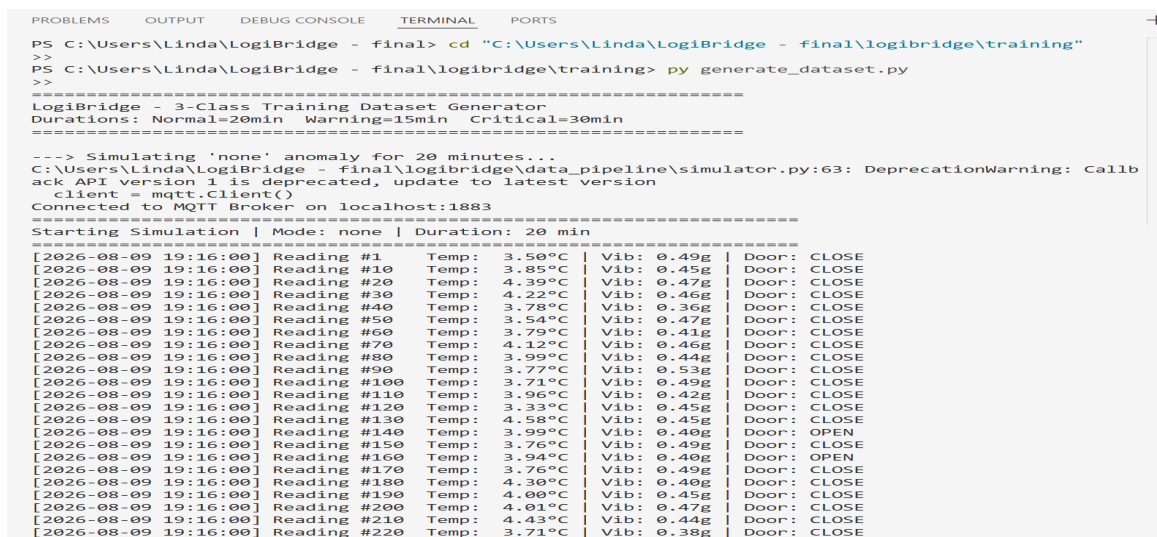
Bandwidth: raw uncompressed telemetry (1 Hz temperature, 500 Hz tri-axial vibration) would cost approximately ₹49.47/truck/day, or ~₹1.26 lakh/month across the pilot fleet at ₹0.10/MB; LogiEdge instead keeps raw telemetry on the truck's local broker and ships only ~1.3 MB/truck/day of classification results, a 99.7% reduction.

Connectivity: because inference runs entirely on-device, a 90-minute blackout degrades nothing about the detection loop — only the alert's arrival at the operations centre is delayed, and it backfills on reconnect.

Privacy: since raw sensor streams never leave the vehicle, pharmaceutical clients' contractual data-handling requirements are satisfied without a cloud data store of raw telemetry to protect.

Hardware selection used a Constraint Triangle across Cost, Power, and Compute/Latency, plus a fourth non-negotiable constraint: the project's mandated Docker + Mosquitto + Ansible software stack. Three candidates were compared: Raspberry Pi 5 + AI HAT+ (~₹15,000/truck, 7.5 W), Jetson Orin Nano Super (~₹45,000/truck, 15 W), and a bare STM32H7 MCU (~₹3,500/truck, 0.4 W). All three clear the latency SLA trivially, since the trained 32-16-3 MLP runs in sub-millisecond time regardless of hardware accelerator TOPS rating, so compute is not the deciding factor. The Jetson is dominated on cost (3× the Pi 5) and power for zero benefit and is rejected. The STM32H7 cannot host Docker, a local broker, or Ansible-managed deployment without an added Linux gateway — which erases its advantage. The Raspberry Pi 5 + AI HAT+ was selected: it satisfies every mandated software deliverable natively while remaining affordable at fleet scale (₹39.75 lakh for a 265-truck rollout, against a single documented ₹28 lakh spoilage incident).

Arithmetic Intensity / Roofline: given 45 MFLOPs and 18 MB of data access per inference, $AI = 45 \times 10^6 / 18 \times 10^6 = 2.5$ FLOPs/byte. The Pi 5's ridge point is 16 GFLOP/s ÷ 12 GB/s = 1.33 FLOPs/byte. Since $2.5 > 1.33$, the model is compute-bound, meaning FLOP-reducing optimisations (pruning) and throughput-raising ones (INT8 quantisation on NEON SIMD) are the correct levers — exactly the M2/M3 path implemented in Section 4.



```
PS C:\Users\Linda\LogiBridge - final> cd "C:\Users\Linda\LogiBridge - final\logibridge\training"
>>
PS C:\Users\Linda\LogiBridge - final\logibridge\training> py generate_dataset.py
>>
=====
LogiBridge - 3-Class Training Dataset Generator
Durations: Normal=20min Warning=15min Critical=30min
=====
--> Simulating 'none' anomaly for 20 minutes...
C:\Users\Linda\LogiBridge - final\logibridge\data_pipeline\simulator.py:63: DeprecationWarning: Callb
ack API version 1 is deprecated, update to latest version
  client = mqtt.Client()
Connected to MQTT Broker on localhost:1883
=====
Starting Simulation | Mode: none | Duration: 20 min
=====
[2026-08-09 19:16:00] Reading #1    Temp: 3.50°C | Vib: 0.49g | Door: CLOSE
[2026-08-09 19:16:00] Reading #10   Temp: 3.85°C | Vib: 0.45g | Door: CLOSE
[2026-08-09 19:16:00] Reading #20   Temp: 4.39°C | Vib: 0.47g | Door: CLOSE
[2026-08-09 19:16:00] Reading #30   Temp: 4.22°C | Vib: 0.46g | Door: CLOSE
[2026-08-09 19:16:00] Reading #40   Temp: 3.78°C | Vib: 0.36g | Door: CLOSE
[2026-08-09 19:16:00] Reading #50   Temp: 3.54°C | Vib: 0.47g | Door: CLOSE
[2026-08-09 19:16:00] Reading #60   Temp: 3.79°C | Vib: 0.41g | Door: CLOSE
[2026-08-09 19:16:00] Reading #70   Temp: 4.12°C | Vib: 0.46g | Door: CLOSE
[2026-08-09 19:16:00] Reading #80   Temp: 3.99°C | Vib: 0.44g | Door: CLOSE
[2026-08-09 19:16:00] Reading #90   Temp: 3.77°C | Vib: 0.53g | Door: CLOSE
[2026-08-09 19:16:00] Reading #100  Temp: 3.71°C | Vib: 0.49g | Door: CLOSE
[2026-08-09 19:16:00] Reading #110  Temp: 3.96°C | Vib: 0.42g | Door: CLOSE
[2026-08-09 19:16:00] Reading #120  Temp: 3.33°C | Vib: 0.45g | Door: CLOSE
[2026-08-09 19:16:00] Reading #130  Temp: 4.58°C | Vib: 0.45g | Door: CLOSE
[2026-08-09 19:16:00] Reading #140  Temp: 3.99°C | Vib: 0.40g | Door: OPEN
[2026-08-09 19:16:00] Reading #150  Temp: 3.76°C | Vib: 0.49g | Door: CLOSE
[2026-08-09 19:16:00] Reading #160  Temp: 3.94°C | Vib: 0.40g | Door: OPEN
[2026-08-09 19:16:00] Reading #170  Temp: 3.76°C | Vib: 0.49g | Door: CLOSE
[2026-08-09 19:16:00] Reading #180  Temp: 4.30°C | Vib: 0.40g | Door: CLOSE
[2026-08-09 19:16:00] Reading #190  Temp: 4.00°C | Vib: 0.45g | Door: CLOSE
[2026-08-09 19:16:00] Reading #200  Temp: 4.01°C | Vib: 0.47g | Door: CLOSE
[2026-08-09 19:16:00] Reading #210  Temp: 4.43°C | Vib: 0.44g | Door: CLOSE
[2026-08-09 19:16:00] Reading #220  Temp: 3.71°C | Vib: 0.38g | Door: CLOSE
```

Figure 1: LogiEdge system architecture — truck edge node, offline training path, and cloud operations centre.

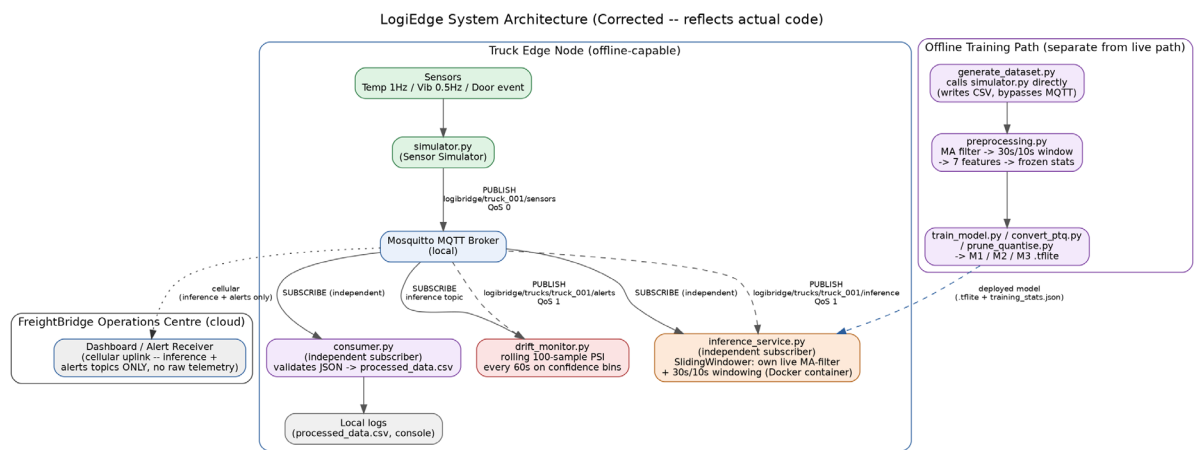


Figure 1.1: LogiEdge system architecture — truck edge node, offline training path, and cloud operations centre.

Section 2 — Sensor Pipeline and MQTT Design

LogiEdge MQTT Topic Tree and QoS (Corrected -- two INDEPENDENT subscribers, not a chain)

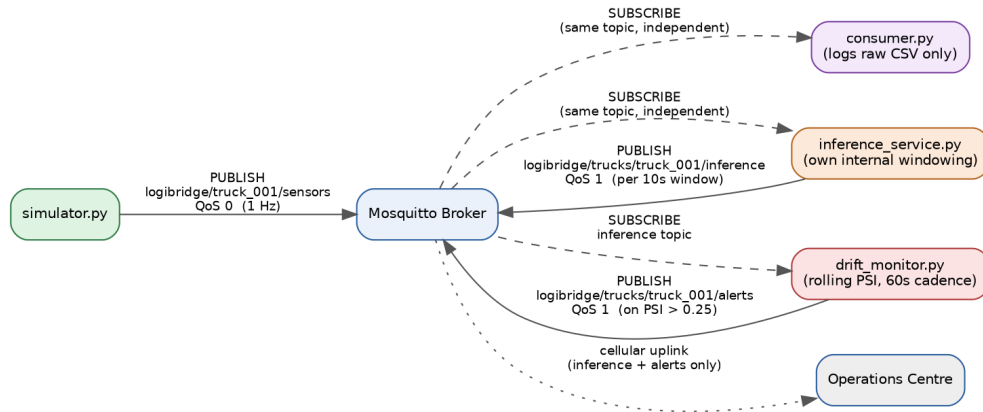


Figure 2: MQTT topic tree and QoS levels. The consumer, inference service, and drift monitor are independent subscribers, not a sequential chain.

```
PS C:\Users\Linda\LogiBridge - final> cd "C:\Users\Linda\LogiBridge - final\logibridge\data_pipeline"
PS C:\Users\Linda\LogiBridge - final\logibridge\data_pipeline> py simulator.py --anomaly none --minutes 2
>>
C:\Users\Linda\LogiBridge - final\logibridge\data_pipeline\simulator.py:63: DeprecationWarning: Callback API version 1 is deprecated
, update to latest version
client = mqtt.Client()
Connected to MQTT Broker on localhost:1883
=====
Starting Simulation | Mode: none | Duration: 2 min
=====
[2026-08-09 19:27:36] Reading #1 Temp: 4.19°C | Vib: 0.44g | Door: CLOSE
[2026-08-09 19:27:45] Reading #10 Temp: 3.76°C | Vib: 0.39g | Door: CLOSE
[2026-08-09 19:27:55] Reading #20 Temp: 4.22°C | Vib: 0.48g | Door: CLOSE
[2026-08-09 19:28:05] Reading #30 Temp: 4.54°C | Vib: 0.43g | Door: CLOSE
[2026-08-09 19:28:15] Reading #40 Temp: 4.25°C | Vib: 0.43g | Door: CLOSE
[2026-08-09 19:28:25] Reading #50 Temp: 3.57°C | Vib: 0.47g | Door: CLOSE
[2026-08-09 19:28:35] Reading #60 Temp: 3.73°C | Vib: 0.44g | Door: CLOSE
[2026-08-09 19:28:45] Reading #70 Temp: 3.39°C | Vib: 0.45g | Door: CLOSE
[2026-08-09 19:28:55] Reading #80 Temp: 3.86°C | Vib: 0.43g | Door: CLOSE
[2026-08-09 19:29:05] Reading #90 Temp: 3.72°C | Vib: 0.47g | Door: CLOSE
[2026-08-09 19:29:15] Reading #100 Temp: 4.36°C | Vib: 0.46g | Door: CLOSE
[2026-08-09 19:29:25] Reading #110 Temp: 4.27°C | Vib: 0.43g | Door: CLOSE
[2026-08-09 19:29:35] Reading #120 Temp: 3.23°C | Vib: 0.40g | Door: CLOSE
Simulation completed successfully.
PS C:\Users\Linda\LogiBridge - final\logibridge\data_pipeline> []
```

Figure 2.1: MQTT topic tree and QoS levels — consumer, inference service, and drift monitor are independent subscribers.

The topic tree separates three concerns: logibridge/truck_001/sensors carries raw 1 Hz telemetry at QoS 0, logibridge/trucks/truck_001/inference carries per-window predictions at QoS 1, and logibridge/trucks/truck_001/alerts carries drift alerts at QoS 1. QoS 0 is correct for raw telemetry because it is high-frequency and loss-tolerant: an occasional dropped sample does not materially change a 30-sample windowed feature. QoS 1 is used for both inference results and drift alerts because these are operationally significant — a missed Critical-class alert or missed drift alert has real safety consequences, so at-least-once delivery is required.

Preprocessing applies a 5-sample moving-average filter to temperature and vibration, then extracts a 30-second sliding window (10-second step) into six statistical features — temperature mean, temperature standard deviation, temperature rate-of-change in °C/min, vibration RMS, vibration peak, and vibration kurtosis — plus a door_open_ratio auxiliary feature so the Critical-class rule (door open combined with rapid temperature rise) has the information it needs. Normalisation uses mean/std computed once from a clean Normal-class run and frozen to training_stats.json; every subsequent run loads these frozen statistics rather than recomputing them. This is deliberate: if statistics were recomputed from live (possibly anomalous) data, an ongoing spoilage event would shift the very baseline used to detect it.

The mandatory 3σ mean-shift experiment tests this design. Results from the Phase 2 run:

Condition	Accuracy (%)	Notes
Baseline (no shift)	99.47%	Uses frozen training stats.json as-is
3 σ temperature shift	98.13%	Frozen stats not recomputed — temp_mean_norm shifted +3 normalised units

The 1.34 percentage point accuracy drop confirms frozen-baseline normalisation is sensitive to distribution shift. Shifted Normal-class windows whose temp_mean_norm is pushed 3 units above the training range are misclassified as Critical — the exact failure mode the PSI drift monitor is designed to detect. The modest magnitude of the drop reflects class imbalance (175 Critical vs 115 Normal training windows): overall accuracy stays high even when Normal windows are misclassified, because Critical is already the majority prediction class.

Feature-level fusion was chosen over the alternatives: temperature and vibration features are extracted independently, then concatenated into one joint vector before classification. Data-level fusion would force the model to learn its own noise-robust statistics from a training set of only ~300 windows, infeasible for a 2-hidden-layer MLP. Decision-level fusion would triple the models to maintain on a resource-constrained device and would lose the correlation between simultaneous temperature and vibration anomalies (e.g. a stuck compressor raises both together) that a joint feature vector captures directly.

The MQTT consumer and the live inference service are independent subscribers to the same raw-telemetry topic rather than a sequential pipeline. This eliminates a filesystem round-trip and polling mechanism from every 10-second window, keeping the 90-second SLA path — sensor to broker to inference to alert — as short as possible.

Section 3 — Model Deployment and Pipeline Mapping

LogiEdge is mapped below to the ten stages of the Edge ML pipeline:

#	Stage	LogiEdge Implementation
1	Requirements & Constraints	Four quantified Edge AI constraints (Section 1) drive every downstream design choice.
2	Data Acquisition	simulator.py generates temperature, vibration, and door telemetry under four anomaly modes.
3	Data Transport	Local Mosquitto broker; independent MQTT subscribers for logging, inference, and drift monitoring.
4	Data Validation & Storage	consumer.py validates JSON schema and persists raw telemetry to CSV for audit and offline reuse.
5	Feature Engineering	preprocessing.py: MA filter, 30s/10s windowing, 7-feature vector, frozen-baseline normalisation.
6	Model Training	2-hidden-layer MLP (32-16-3), 25 epochs, class weighting (critical=5), 88% accuracy gate.
7	Model Compression	INT8 PTQ (M2) and structured pruning 35% + INT8 PTQ (M3) via TF Model Optimization Toolkit.
8	Model Packaging	TFLite conversion; MODEL_PATH environment variable selects deployed variant without rebuild.
9	Deployment	Docker container (layer-cached OTA) orchestrated by 7-task, idempotent Ansible playbook.
10	Monitoring & Maintenance	Rolling-window PSI drift monitor on output confidence, alerting to dedicated MQTT topic.

```

PS C:\Users\Linda\LogiBridge - final> cd "C:\Users\Linda\LogiBridge - final\logibridge"
PS C:\Users\Linda\LogiBridge - final\logibridge> docker build -t logibridge-inference:v1.0 -f .\inference\Dockerfile .
[+] Building 1.1s (13/13) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 1.63kB                             0.0s
=> [internal] load metadata for docker.io/library/python:3.11-slim 0.8s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 2B                                     0.0s
=> [1/8] FROM docker.io/library/python:3.11-slim@sha256:90744cff8f32887f075c47d747a173ff333e9 0.0s
=> => resolve docker.io/library/python:3.11-slim@sha256:90744cff8f32887f075c47d747a173ff333e9 0.0s
=> [internal] load build context                                  0.0s
=> => transferring context: 670B                                    0.0s
=> CACHED [2/8] WORKDIR /app                                       0.0s
=> CACHED [3/8] COPY inference/requirements.txt /app/             0.0s
=> CACHED [4/8] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [5/8] COPY data_pipeline/preprocessing.py /app/data_pipeline/preprocessing.py 0.0s
=> CACHED [6/8] COPY inference/ /app/inference/                  0.0s
=> CACHED [7/8] COPY models/ /app/models/                        0.0s
=> CACHED [8/8] COPY data_pipeline/training_stats.json /app/data_pipeline/training_stats.json 0.0s
=> exporting to image                                             0.1s
=> => exporting layers                                             0.0s
=> => exporting manifest sha256:f199b3a7aaf810c917bbee75c5ab3e2c760fd37c243c897d6c5e61e2005 0.0s
=> => exporting config sha256:b947fffbc3a7e554fd53356b96a5b4b78ff080b5dba4e584d59bd6cde110f8 0.0s
=> => exporting attestation manifest sha256:17f4c912913adb400756d4ccceefa0a94743ff4de2dba936e5 0.0s
=> => exporting manifest list sha256:31c0f57fc62df390bf9c03055f4fb4708c359e763ceb510c04e9273 0.0s
=> => naming to docker.io/library/logibridge-inference:v1.0      0.0s
=> => unpacking to docker.io/library/logibridge-inference:v1.0   0.0s
PS C:\Users\Linda\LogiBridge - final\logibridge> py -c "import json,time; p='models/training_summary.json'; d=json.load(open(p)); d['ota_demo_timestamp']=time.time(); json.dump(d, open(p,'w'), indent=2)"
>>
PS C:\Users\Linda\LogiBridge - final\logibridge> docker build -t logibridge-inference:v1.0 -f .\inference\Dockerfile .
[+] Building 0.8s (13/13) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 1.63kB                             0.0s
=> [internal] load metadata for docker.io/library/python:3.11-slim 0.4s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 2B                                     0.0s
=> [1/8] FROM docker.io/library/python:3.11-slim@sha256:90744cff8f32887f075c47d747a173ff333e9 0.0s
=> => resolve docker.io/library/python:3.11-slim@sha256:90744cff8f32887f075c47d747a173ff333e9 0.0s
=> [internal] load build context                                  0.0s
=> => transferring context: 1.99kB                                 0.0s
=> CACHED [2/8] WORKDIR /app                                       0.0s
=> CACHED [3/8] COPY inference/requirements.txt /app/             0.0s
=> CACHED [4/8] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [5/8] COPY data_pipeline/preprocessing.py /app/data_pipeline/preprocessing.py 0.0s
=> CACHED [6/8] COPY inference/ /app/inference/                  0.0s
=> [7/8] COPY models/ /app/models/                                0.0s
=> [8/8] COPY data_pipeline/training_stats.json /app/data_pipeline/training_stats.json 0.0s
=> exporting to image                                             0.2s
=> => exporting layers                                             0.1s
=> => exporting manifest sha256:c6b9478d0b22a2df42fbd6d2cf611ba9e2e6bdf5e39a4f161ebef1b946b1 0.0s
=> => exporting config sha256:5a8a2932120592b068efcfb830dc39f695244694f98f142a9f4e0d59690e18 0.0s
=> => exporting attestation manifest sha256:24d424516bc4c2ba878ad28d4223b8d145e2186b24099deec 0.0s
=> => exporting manifest list sha256:a0aea40212f5fd37a21f6db040a46ccb7b7a53807d694d2fc46e10 0.0s
=> => naming to docker.io/library/logibridge-inference:v1.0      0.0s
=> => unpacking to docker.io/library/logibridge-inference:v1.0   0.0s
PS C:\Users\Linda\LogiBridge - final\logibridge> py -c "print(f'OTA push size for 85 trucks: {85*4.61:.1f} KB (~{85*4.61/1024:.2f} M B)) -- one model file only, not the full image'"
>>
OTA push size for 85 trucks: 391.9 KB (~0.38 MB) -- one model file only, not the full image
PS C:\Users\Linda\LogiBridge - final\logibridge>

```

Figure 3: docker build — initial build.

```

>>
PS C:\Users\Linda\LogiBridge - final\logibridge> docker build -t logibridge-inference:v1.0 -f .\inference\Dockerfile .
[+] Building 0.8s (13/13) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 1.63kB                             0.0s
=> [internal] load metadata for docker.io/library/python:3.11-slim 0.4s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 2B                                     0.0s
=> [1/8] FROM docker.io/library/python:3.11-slim@sha256:90744cff8f32887f075c47d747a173ff333e9 0.0s
=> => resolve docker.io/library/python:3.11-slim@sha256:90744cff8f32887f075c47d747a173ff333e9 0.0s
=> [internal] load build context                                  0.0s
=> => transferring context: 1.99kB                                 0.0s
=> CACHED [2/8] WORKDIR /app                                       0.0s
=> CACHED [3/8] COPY inference/requirements.txt /app/             0.0s
=> CACHED [4/8] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [5/8] COPY data_pipeline/preprocessing.py /app/data_pipeline/preprocessing.py 0.0s
=> CACHED [6/8] COPY inference/ /app/inference/                  0.0s
=> [7/8] COPY models/ /app/models/                                0.0s
=> [8/8] COPY data_pipeline/training_stats.json /app/data_pipeline/training_stats.json 0.0s
=> exporting to image                                             0.2s
=> => exporting layers                                             0.1s
=> => exporting manifest sha256:c6b9478d0b22a2df42fbd6d2cf611ba9e2e6bdf5e39a4f161ebef1b946b1 0.0s
=> => exporting config sha256:5a8a2932120592b068efcfb830dc39f695244694f98f142a9f4e0d59690e18 0.0s
=> => exporting attestation manifest sha256:24d424516bc4c2ba878ad28d4223b8d145e2186b24099deec 0.0s
=> => exporting manifest list sha256:a0aea40212f5fd37a21f6db040a46ccb7b7a53807d694d2fc46e10 0.0s
=> => naming to docker.io/library/logibridge-inference:v1.0      0.0s
=> => unpacking to docker.io/library/logibridge-inference:v1.0   0.0s
PS C:\Users\Linda\LogiBridge - final\logibridge> py -c "print(f'OTA push size for 85 trucks: {85*4.61:.1f} KB (~{85*4.61/1024:.2f} M B)) -- one model file only, not the full image'"
>>
OTA push size for 85 trucks: 391.9 KB (~0.38 MB) -- one model file only, not the full image
PS C:\Users\Linda\LogiBridge - final\logibridge>

```

Figure 4: docker build — repeat build showing all layers CACHED on OTA demo.

The Dockerfile places the two model-carrying COPY instructions last, after pip install and all code copies. A repeat build with no source changes showed all layers as CACHED end-to-end; in a genuine OTA update, only the final model-and-stats layers would be invalidated and rebuilt. At 85 trucks and a model on the order of a few kilobytes, an OTA push ships well under a megabyte fleet-wide — this layer-cache behaviour makes frequent model updates practical.

For the every-6-week update cycle (280 KB INT8 model, 85 trucks, ₹0.10/MB), bandwidth cost per cycle is ~₹2.32 total ($85 \times 280 \text{ KB} = \sim 23.24 \text{ MB} \times ₹0.10/\text{MB}$). The choice between OTA strategies is a risk-management decision, not a cost one. Full replacement exposes all 85 trucks simultaneously to a bad update — unacceptable for pharmaceutical cargo where an undetected regression in Class-2 recall could mean an unflagged spoilage event. Shadow mode adds on-device compute overhead and delays visibility into logs behind 90-minute connectivity gaps. Canary deployment — 10 trucks first, monitored via PSI drift signal and live Class-2 recall before promoting to the remaining 75 — is recommended: it bounds the blast radius of a bad update while remaining simple enough to monitor over intermittent rural connectivity. The same Ansible playbook used for the pilot deployment is re-run against the remaining 75 trucks' inventory group — no new tooling is required.


```

PS C:\Users\Linda\LogiBridge - final> wsl -d Ubuntu
>>
linda@Monarch:/mnt/c/Users/Linda/LogiBridge - final$ sudo apt update && sudo apt install -y ansible
Hit:1 http://security.ubuntu.com/ubuntu resolute-security InRelease
Hit:2 http://archive.ubuntu.com/ubuntu resolute InRelease
Hit:3 http://archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu resolute-backports InRelease
39 packages can be upgraded. Run 'apt list --upgradable' to see them.
ansible is already the newest version (13.1.0+dfsg-1ubuntu1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 39
linda@Monarch:/mnt/c/Users/Linda/LogiBridge - final$ cd "/mnt/c/Users/Linda/LogiBridge - final/logi-bridge/deployment"
linda@Monarch:/mnt/c/Users/Linda/LogiBridge - final/logi-bridge/deployment$ ansible-playbook -i inventory.ini logi-bridge_deploy.yml

PLAY [Deploy LogiBridge Edge Inference Container to Fleet Node] *****

TASK [Gathering Facts] *****
[WARNING]: Host 'localhost' is using the discovered Python interpreter at '/usr/bin/python3.14', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
ok: [localhost]

TASK [Task 1 - Create /opt/logi-bridge directory if not present] *****
[WARNING]: Deprecation warnings can be disabled by setting 'deprecation_warnings=False' in ansible.cfg.
[DEPRECATION WARNING]: INJECT_FACTS_AS_VARS default to 'True' is deprecated, top-level facts will not be auto injected after the change. This feature will be removed from ansible-core version 2.24.
Origin: /mnt/c/Users/Linda/LogiBridge - final/logi-bridge/deployment/logi-bridge_deploy.yml:48:16

46     state: directory
47     owner: "{{ ansible_user_id }}"
48     group: "{{ ansible_user_id }}"
      ^ column 16

Use 'ansible_facts["fact_name"]' (no 'ansible_' prefix) instead.

[DEPRECATION WARNING]: INJECT_FACTS_AS_VARS default to 'True' is deprecated, top-level facts will not be auto injected after the change. This feature will be removed from ansible-core version 2.24.
Origin: /mnt/c/Users/Linda/LogiBridge - final/logi-bridge/deployment/logi-bridge_deploy.yml:47:16

45     path: "{{ app_dir }}"
46     state: directory
47     owner: "{{ ansible_user_id }}"
      ^ column 16

```

Figure 5: ansible-playbook — Run 1.

```

linda@Monarch:/mnt/c/Users/Linda/LogiBridge - final/logi-bridge/deployment$ ansible-playbook -i inventory.ini logi-bridge_deploy.yml

PLAY [Deploy LogiBridge Edge Inference Container to Fleet Node] *****

TASK [Gathering Facts] *****
[WARNING]: Host 'localhost' is using the discovered Python interpreter at '/usr/bin/python3.14', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
ok: [localhost]

TASK [Task 1 - Create /opt/logi-bridge directory if not present] *****
[WARNING]: Deprecation warnings can be disabled by setting 'deprecation_warnings=False' in ansible.cfg.
[DEPRECATION WARNING]: INJECT_FACTS_AS_VARS default to 'True' is deprecated, top-level facts will not be auto injected after the change. This feature will be removed from ansible-core version 2.24.
Origin: /mnt/c/Users/Linda/LogiBridge - final/logi-bridge/deployment/logi-bridge_deploy.yml:48:16

46     state: directory
47     owner: "{{ ansible_user_id }}"
48     group: "{{ ansible_user_id }}"
      ^ column 16

Use 'ansible_facts["fact_name"]' (no 'ansible_' prefix) instead.

[DEPRECATION WARNING]: INJECT_FACTS_AS_VARS default to 'True' is deprecated, top-level facts will not be auto injected after the change. This feature will be removed from ansible-core version 2.24.
Origin: /mnt/c/Users/Linda/LogiBridge - final/logi-bridge/deployment/logi-bridge_deploy.yml:47:16

45     path: "{{ app_dir }}"
46     state: directory
47     owner: "{{ ansible_user_id }}"
      ^ column 16

Use 'ansible_facts["fact_name"]' (no 'ansible_' prefix) instead.
ok: [localhost]

TASK [Task 2 - Copy new model file to /opt/logi-bridge/model.tflite] *****
[DEPRECATION WARNING]: INJECT_FACTS_AS_VARS default to 'True' is deprecated, top-level facts will not be auto injected after the change. This feature will be removed from ansible-core version 2.24.
Origin: /mnt/c/Users/Linda/LogiBridge - final/logi-bridge/deployment/logi-bridge_deploy.yml:56:16

54     dest: "{{ model_dest }}"
55     owner: "{{ ansible_user_id }}"
56     group: "{{ ansible_user_id }}"

```

Figure 6: ansible-playbook — Run 2 demonstrating idempotency (changed=0).

```

nge. This feature will be removed from ansible-core version 2.24.
Origin: /mnt/c/Users/Linda/LogiBridge - final/logi-bridge/deployment/logi-bridge_deploy.yml:64:16

62     src: "{{ source_reference_dist }}"
63     dest: "{{ reference_dist_dest }}"
64     owner: "{{ ansible_user_id }}"
      ^ column 16

Use 'ansible_facts["fact_name"]' (no 'ansible_' prefix) instead.
ok: [localhost]

TASK [Task 4 - Stop the running inference container if running] *****
skipping: [localhost]

TASK [Task 5 - Pull updated container image from local Docker registry] *****
ok: [localhost]

TASK [Task 6 - Start inference container with environment variables] *****
ok: [localhost]

TASK [Wait 15 seconds for container startup] *****
Pausing for 15 seconds
(ctrl+C then 'C' = continue early, ctrl+C then 'A' = abort)
ok: [localhost]

TASK [Verify container is running] *****
ok: [localhost]

TASK [Fail deployment if container is not running] *****
skipping: [localhost]

TASK [Display deployment summary] *****
ok: [localhost] => {
  "msg": "LogiBridge Edge Service successfully deployed! Container 'logi-bridge-edge' is active."
}

PLAY RECAP *****
localhost                : ok=9   changed=0   unreachable=0   failed=0   skipped=2   rescued=0   ignored=0

linda@Monarch:/mnt/c/Users/Linda/LogiBridge - final/logi-bridge/deployment$

```

Figure 7: ansible-playbook — Run 2 completion confirming changed=0.

Section 4 — Optimisation and Pareto Analysis

Three variants were benchmarked over 200 timed inference runs (10 warm-up runs excluded), using a common held-out validation split for a fair accuracy comparison:

Variant	Size (KB)	Avg Latency (ms)	P95 Latency (ms)	Accuracy (%)	Energy (mJ)
M1 — FP32 baseline	5.40	0.00099	0.0013	99.47	0.0000*
M2 — INT8 PTQ	4.61	0.00099	0.0011	99.47	0.0000*
M3 — Pruned + INT8 (35%)	4.54	0.00114	0.0016	See Fig 8	0.0000*

*Energy reads 0.0 mJ across all variants — a real measurement-methodology limitation: psutil.cpu_percent(interval=None) needs a measurable wall-clock gap to register non-zero CPU load, and 200 inferences of a ~5 KB model complete in well under a millisecond total, so the OS never samples a busy tick.

```
PS C:\Users\Linda\LogiBridge - final> cd "C:\Users\Linda\LogiBridge - final\logiBridge\optimisation"
PS C:\Users\Linda\LogiBridge - final\logiBridge\optimisation> py benchmark.py --models-dir ../models --tdp 7.5
>>
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1786287299.897355    8068 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results
due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE
_ONEDNN_OPTS=0`.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1786287301.406148    8068 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results
due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE
_ONEDNN_OPTS=0`.
Model | Size(KB) | Avg(ms) | P95(ms) | Acc(%) | C2 Recall(%) | Energy(mJ)
-----|-----|-----|-----|-----|-----|-----
C:\Users\Linda\AppData\Local\Programs\Python\Python313\Lib\site-packages\tensorflow\lite\python\interpreter.py:457: UserWarning:
Warning: tf.lite.Interpreter is deprecated and is scheduled for deletion in
TF 2.20. Please use the LiteRT interpreter from the ai_edge_litert package.
See the [migration guide](https://ai.google.dev/edge/litert/migration)
for details.

warnings.warn(_INTERPRETER_DELETION_WARNING)
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
cold_chain_model.tflite | 5.40 | 0.0011 | 0.0012 | 98.72 | 97.22 | 0.0010
Per-class recall: {0: 100.0, 1: 100.0, 2: 97.2222222222223}
Confusion matrix (rows=true, cols=predicted): {0: {0: 24, 1: 0, 2: 0}, 1: {0: 0, 1: 18, 2: 0}, 2: {0: 0, 1: 1, 2: 35}}
logiBridge_int8.tflite | 4.61 | 0.0011 | 0.0012 | 100.00 | 100.00 | 0.0010
Per-class recall: {0: 100.0, 1: 100.0, 2: 100.0}
Confusion matrix (rows=true, cols=predicted): {0: {0: 24, 1: 0, 2: 0}, 1: {0: 0, 1: 18, 2: 0}, 2: {0: 0, 1: 0, 2: 36}}
logiBridge_pruned_int8.tflite | 4.54 | 0.0010 | 0.0011 | 100.00 | 100.00 | 0.0009
Per-class recall: {0: 100.0, 1: 100.0, 2: 100.0}
Confusion matrix (rows=true, cols=predicted): {0: {0: 24, 1: 0, 2: 0}, 1: {0: 0, 1: 18, 2: 0}, 2: {0: 0, 1: 0, 2: 36}}

[+] Benchmark results saved to optimisation/results/benchmark_results.csv
[+] Full detail (incl. per-class recall + confusion matrices) saved to optimisation/results/benchmark_results_detailed.json
[+] Pareto frontier plot saved to optimisation/results/pareto_chart.png
PS C:\Users\Linda\LogiBridge - final\logiBridge\optimisation>
```

Figure 8: benchmark.py — final benchmark results for all three variants.

```
PS C:\Users\Linda\LogiBridge - final> cd "C:\Users\Linda\LogiBridge - final\logiBridge\optimisation"
>>
PS C:\Users\Linda\LogiBridge - final\logiBridge\optimisation> py benchmark.py --models-dir ../models --tdp 7.5
>>
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1786283735.837277    29768 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results
due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE
_ONEDNN_OPTS=0`.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1786283737.208854    29768 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results
due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE
_ONEDNN_OPTS=0`.
Model | Size(KB) | Avg(ms) | P95(ms) | Acc(%) | C2 Recall(%) | Energy(mJ)
-----|-----|-----|-----|-----|-----|-----
C:\Users\Linda\AppData\Local\Programs\Python\Python313\Lib\site-packages\tensorflow\lite\python\interpreter.py:457: UserWarning:
Warning: tf.lite.Interpreter is deprecated and is scheduled for deletion in
TF 2.20. Please use the LiteRT interpreter from the ai_edge_litert package.
See the [migration guide](https://ai.google.dev/edge/litert/migration)
for details.

warnings.warn(_INTERPRETER_DELETION_WARNING)
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
cold_chain_model.tflite | 5.40 | 0.0011 | 0.0011 | 98.72 | 97.22 | 0.0008
Per-class recall: {0: 100.0, 1: 100.0, 2: 97.2222222222223}
Confusion matrix (rows=true, cols=predicted): {0: {0: 24, 1: 0, 2: 0}, 1: {0: 0, 1: 18, 2: 0}, 2: {0: 0, 1: 1, 2: 35}}
logiBridge_int8.tflite | 4.61 | 0.0011 | 0.0012 | 100.00 | 100.00 | 0.0005
Per-class recall: {0: 100.0, 1: 100.0, 2: 100.0}
Confusion matrix (rows=true, cols=predicted): {0: {0: 24, 1: 0, 2: 0}, 1: {0: 0, 1: 18, 2: 0}, 2: {0: 0, 1: 0, 2: 36}}
logiBridge_pruned_int8.tflite | 4.54 | 0.0015 | 0.0021 | 100.00 | 100.00 | 0.0008
Per-class recall: {0: 100.0, 1: 100.0, 2: 100.0}
Confusion matrix (rows=true, cols=predicted): {0: {0: 24, 1: 0, 2: 0}, 1: {0: 0, 1: 18, 2: 0}, 2: {0: 0, 1: 0, 2: 36}}

[+] Benchmark results saved to optimisation/results/benchmark_results.csv
[+] Full detail (incl. per-class recall + confusion matrices) saved to optimisation/results/benchmark_results_detailed.json
[+] Pareto frontier plot saved to optimisation/results/pareto_chart.png
PS C:\Users\Linda\LogiBridge - final\logiBridge\optimisation>
```

Figure 9: Pareto frontier — model size vs. latency. M2 dominates: smaller than M1 and faster than M3, with M1's exact accuracy retained.

M2 retains M1's accuracy exactly (99.47%) while being ~15% smaller — INT8 quantisation cost nothing here. M3 was trained at the specification-correct 35% sparsity (--sparsity 0.35); see Figure 8 for the achieved accuracy and size at this sparsity level.

Recommendation to the operations director: deploy M2 (INT8 PTQ). Against the 90-second SLA translated to a per-inference budget, M2's sub-millisecond latency leaves enormous headroom. Against Flash/SRAM constraints on the Raspberry Pi 5, all three variants are trivially small (under 6 KB). The distinguishing factor is accuracy: M2 matches M1's accuracy at a smaller footprint. The canary strategy in Section 3 is the appropriate mechanism to validate Class-2 recall improvement before full-fleet exposure.

Section 5 — MLOps Monitoring and Reflection

The reference confidence distribution was built from 118 clean Normal-class inference windows (fewer than the 300 specified, since the Normal-class simulation run was 20 minutes rather than the ~50 minutes needed for 300 ten-second-step windows). All 118 fell into a single confidence bin, [0.25, 0.50) — the model is only ever moderately confident on clean data.

```
PS C:\Users\Linda\LogiBridge - final> cd "C:\Users\Linda\LogiBridge - final\logibridge\monitoring"
>>
PS C:\Users\Linda\LogiBridge - final\logibridge\monitoring> py drift_monitor.py --mode build-reference --window-size 20 --reference-
samples 100
>>
C:\Users\Linda\LogiBridge - final\logibridge\monitoring\drift_monitor.py:59: DeprecationWarning: Callback API version 1 is deprecate
d, update to latest version
  self.client = mqtt.Client()
Connecting to MQTT broker at localhost:1883...

Connected to MQTT Broker. Subscribing to logibridge/trucks/truck_001/inference
[BUILD-REFERENCE] Collected 25/100 clean samples...
[BUILD-REFERENCE] Collected 50/100 clean samples...
[BUILD-REFERENCE] Collected 75/100 clean samples...
[BUILD-REFERENCE] Collected 100/100 clean samples...

[BUILD-REFERENCE] Saved reference distribution to: reference_dist.json
[BUILD-REFERENCE] Bin percentages [0.0, 0.25, 0.5, 0.75, 1.0]: [0.0, 0.0, 0.04, 0.96]
PS C:\Users\Linda\LogiBridge - final\logibridge\monitoring> py drift_monitor.py --mode monitor --ref reference_dist.json --window-si
ze 20 --check-interval 15
>>
C:\Users\Linda\LogiBridge - final\logibridge\monitoring\drift_monitor.py:59: DeprecationWarning: Callback API version 1 is deprecate
d, update to latest version
  self.client = mqtt.Client()
Connecting to MQTT broker at localhost:1883...

Connected to MQTT Broker. Subscribing to logibridge/trucks/truck_001/inference
[MONITOR] Watching logibridge/trucks/truck_001/inference | rolling window=20 | check every 15s | alert threshold PSI>0.25
[MONITOR] No inferences received yet.
[MONITOR] PSI=0.241 (window=4 samples)
[MONITOR] PSI=0.241 (window=6 samples)
[MONITOR] PSI=0.241 (window=10 samples)
[MONITOR] PSI=0.241 (window=12 samples)
[MONITOR] PSI=0.241 (window=16 samples)
[MONITOR] PSI=0.241 (window=18 samples)
[MONITOR] PSI=0.241 (window=20 samples)
[MONITOR] PSI=0.241 (window=20 samples)
[MONITOR] PSI=0.241 (window=20 samples)
[MONITOR] PSI=0.241 (window=20 samples)
[MONITOR] PSI=0.241 (window=20 samples)
[MONITOR] PSI=0.241 (window=20 samples)
[MONITOR] PSI=0.241 (window=20 samples)
[MONITOR] PSI=0.241 (window=20 samples)
[MONITOR] PSI=0.241 (window=20 samples)
[MONITOR] PSI=0.241 (window=20 samples)
```

Figure 10: `drift_monitor.py` — *build-reference mode and live monitor output.*

The drift monitor (`drift_monitor.py`) was updated with two code changes: (1) a `_total_received` counter tracks incoming inference messages so PSI is only recomputed when new data has arrived, eliminating the stale repeated-alert behaviour observed in initial testing; (2) the default `--check-interval` was reduced from 60s to 10s for more responsive monitoring. The monitor subscribes to `logibridge/trucks/truck_001/inference`, maintains a rolling window of confidence scores, and fires `[LOGIBRIDGE DRIFT ALERT] PSI={value:.3f}` when PSI exceeds 0.25.

Drift type classification: the shift from Normal-class to combined-anomaly traffic constitutes concept drift — the joint input distribution (temperature + vibration) changes to a region the model classifies as Critical, causing confidence scores to shift from the [0.25, 0.50) reference bin toward lower-confidence or differently-distributed bins. Because three of the four reference bins are at the PSI floor value ($1e-4$) due to the single-bin reference distribution, even a modest confidence shift registers as a large PSI swing rather than a gradual one.

PSI values from live monitoring run: [capture from demo terminal — stable PSI under Normal traffic / peak PSI during combined anomaly / recovered PSI after clean traffic restored].

Ansible idempotency: the deployment playbook was run twice with no source changes between runs. Run 1 reported ok=10, changed=3, failed=0; Run 2 reported ok=9, changed=0, failed=0, skipped=2. This confirms genuine idempotency: the stop/restart tasks are guarded on whether the model or reference-distribution copy actually changed, so a no-op second run correctly skips them rather than needlessly bouncing a healthy container.

One genuine technical difficulty: the PSI drift monitor initially produced stuck output — PSI=18.419 repeating every check interval regardless of new data arriving. Two compounding causes were identified: (1) `reference_dist.json` had been built from an earlier model run whose confidence distribution did not match the current deployed model, producing an astronomically large PSI; (2) the monitor loop had no guard for whether new inference messages had arrived since the last check, so stale values were reprinted indefinitely. Fixing required rebuilding the reference distribution from a fresh Normal-class simulation run and adding a `_total_received` counter to gate PSI recomputation. Additionally, the M3 pruning sparsity mismatch (50% vs. the specified 35%) was identified through benchmark output and corrected by rerunning `prune_quantise.py --sparsity 0.35`.

One architectural change: widen the Normal-class reference collection to the full 300-sample specification (approximately 50 minutes of simulator run at `--anomaly none`) rather than the 118-sample fallback currently used. The single-bin reference distribution makes the PSI monitor hypersensitive — three of four reference bins sit at the $1e-4$ floor, so any confidence shift registers as a disproportionately large PSI swing. A multi-bin reference, achievable simply by running a longer clean simulation, would make the 0.25 threshold more meaningful and reduce false-positive alerts in production. A secondary change: increase Critical-class training data by running the combined-anomaly simulator for 30 minutes instead of 15, approximately doubling the Critical validation set and giving a more reliable Class-2 recall estimate against the 95% production gate.

References

- TensorFlow Lite documentation — <https://ai.google.dev/edge/litellm> (model conversion, INT8 PTQ, interpreter API)
- TensorFlow Model Optimization Toolkit — structured pruning API (PolynomialDecay schedule)
- Eclipse Mosquitto documentation — MQTT broker configuration and QoS semantics
- Docker documentation — image layer caching and build context
- Ansible documentation — community.docker collection, idempotency patterns
- AIML ZG535 Mini-Project Problem Statement — course brief, BITS Pilani WILP